

WEEK 3 CODING EXERCISE

Data Collection, Data Cleaning, and Exploratory Data Analysis for ML/NLP

Platform: Jupyter Notebook, Google Colab, or VS Code | Dataset: Synthetic/local student dataset | Purpose: Prepare data for subsequent baseline ML/NLP modeling. Filename: Week3-Data-LN-FN.ipynb

This exercise introduces the practical data preparation # workflow that must be completed before developing an # ML/NLP model.

WORKFLOW: Data Collection -> Dataset Loading -> Data Inspection -> Data Quality Assessment -> Data Cleaning -> Exploratory Data Analysis (EDA) -> Feature and Target Identification -> NLP Text Preparation -> Ethical Assessment -> Clean Dataset -> Ready for Baseline Modeling

Part 1 — Import the Required Libraries

STUDENT GUIDE

Import the Python libraries needed for data manipulation, numerical computation, and visualization. pandas is used for tabular data, numpy for numerical operations and missing values, and matplotlib for graphs. These are common tools in ML/NLP data preparation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries successfully imported!")
```

Part 2 — Create a Local Practice Dataset

STUDENT GUIDE

We use synthetic data so students can safely practice without exposing real student information. The scenario combines a supervised ML task—predicting Academic_Risk—with an NLP task—analyzing student comments. The dataset intentionally contains missing values, duplicates, inconsistent categories, an outlier/data-entry error, and text variations.

DATASET SCENARIO: Imagine that a researcher wants to develop a system that predicts whether a student may be academically at risk. At the same time, the researcher wants to analyze student comments using Natural Language Processing (NLP). Therefore, our dataset contains both: NUMERICAL/CATEGORICAL DATA, Age, Sex, Program, Study_Hours, Attendance, GPA.

TARGET VARIABLE, Academic_Risk

TEXTUAL VARIABLE, Comment.

IMPORTANT: The dataset intentionally contains data-quality problems. These problems are included so that you can practice # detecting and correcting them.

```
data = {
  "Student_ID": [
    "BU001", "BU002", "BU003", "BU004", "BU005",
    "BU006", "BU007", "BU008", "BU009", "BU010",
    "BU011", "BU012", "BU013", "BU014", "BU015",
    "BU003", "BU008", "BU012"
  ],
  "Age": [
    18, 19, 20, 21, 19, 20, 18, 45, 21, np.nan,
    20, 19, 22, 21, 19, 20, 45, 19
  ]
}
```

Week 3 Coding Exercise — ML/NLP Data Preparation

```
,
"Sex": [
    "Female", "Male", "female", "Male", "F",
    "Male", "Female", "Female", "male", "Female",
    "FEMALE", "Male", "Female", "Male", "Female",
    "female", "Female", "Male"
],
"Program": [
    "BSCS", "BSIT", "BSCS", "BSIS", "BSIT",
    "BSCS", "BSIS", "BSCS", "BSIT", "BSIS",
    "BSCS", "BSIT", "BSCS", "BSIS", "BSIT",
    "BSCS", "BSCS", "BSIT"
],
"Study_Hours": [
    5, 8, 3, 6, 4, 7, 2, 200, 5, np.nan,
    6, 8, 4, 5, 3, 3, 200, 8
],
"Attendance": [
    95, 90, 75, 88, 82, 91, 70, 98, 85, 80,
    93, 89, 78, 92, 74, 75, 98, 89
],
"GPA": [
    1.50, 1.75, 2.25, 1.90, 2.50, 1.60, 2.75, 1.25,
    2.00, np.nan, 1.80, 1.70, 2.40, 1.85, 3.20,
    2.25, 1.25, 1.70
],
"Academic_Risk": [
    "No", "No", "Yes", "No", "Yes", "No", "Yes", "No",
    "No", "Yes", "No", "No", "Yes", "No", "Yes",
    "Yes", "No", "No"
],
"Comment": [
    "The lessons are very helpful.",
    "I enjoy the laboratory activities.",
    "The lessons are difficult and confusing.",
    "The teacher explains the topics clearly.",
    "I have difficulty understanding programming.",
    "The activities are interesting.",
    "I struggle with the requirements.",
    "The course is very helpful!!!",
    "The laboratory exercises are useful.",
    "I am having problems with my studies.",
    "The lessons are GOOD.",
    "I like the practical activities.",
    "The topics are difficult.",
    "The instructor provides clear explanations.",
    "I need more time to understand the lessons.",
    "The lessons are difficult and confusing.",
    "The course is very helpful!!!",
    "I like the practical activities."
]
}

df = pd.DataFrame(data)

print("Dataset successfully created!")
```

Part 3 — Display the Dataset

STUDENT GUIDE

Inspect the actual records before cleaning. Look for missing values, duplicate records, inconsistent spelling, unusual numerical values, and differences in text formatting.

df

Question: What problems can you identify just by looking at the data?

Part 4 — Determine the Number of Rows and Columns

STUDENT GUIDE

df.shape returns the dimensions of the dataset. The first value is the number of records and the second is the number of variables. Expected result: 18 records and 9 variables.

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

Part 5 — View the First Records

STUDENT GUIDE

head() displays the first five records. This is one of the first commands researchers use after loading a dataset. Identify the different types of variables you see.

```
df.head()
```

Part 6 — View the Last Records

STUDENT GUIDE

tail() displays the last five records. It is useful for checking that the dataset was loaded correctly and for inspecting records at the end of the file.

```
df.tail()
```

QUESTION: What types of variables can you identify?

Part 7 — Display Column Names

STUDENT GUIDE

We need to know the variables in the dataset because they will later be classified as features, target, text, or identifiers.

```
print("Variables in the dataset:")
print(df.columns.tolist())
```

Part 8 — Inspect Data Types and Missing Values

STUDENT GUIDE

info() shows column names, non-null counts, and data types. Data types matter because numerical, categorical, and text variables require different preprocessing methods. Identify which columns contain missing values.

```
df.info()
```

QUESTION: Which variables contain missing values?

Part 9 — Generate Descriptive Statistics

STUDENT GUIDE

describe() summarizes numerical variables using count, mean, standard deviation, minimum, quartiles, and maximum. Look especially at Study_Hours and identify an unusual value.

```
df.describe()
```

QUESTION: What do you notice about Study_Hours? Is there a value that appears unusual?

Part 10 — Check Missing Values

STUDENT GUIDE

Missing values may affect statistical analysis, visualization, and machine learning. `isnull().sum()` counts missing values in each column.

```
missing_values = df.isnull().sum()

print("Missing values per column:")
print(missing_values)
```

Part 11 — Calculate Percentage of Missing Values

STUDENT GUIDE

A percentage gives more context than a raw count. It helps researchers decide how serious missingness is and whether a variable needs imputation or further investigation.

```
missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_report = pd.DataFrame({
    "Missing_Count": df.isnull().sum(),
    "Missing_Percentage": missing_percentage
})

missing_report
```

Part 12 — Visualize Missing Values

STUDENT GUIDE

A bar chart makes missing-data patterns easier to see. Visualization is part of data-quality assessment and EDA.

```
missing_values = df.isnull().sum()
missing_values = missing_values[missing_values > 0]

plt.figure(figsize=(8, 5))
missing_values.plot(kind="bar")
plt.title("Missing Values by Variable")
plt.xlabel("Variable")
plt.ylabel("Number of Missing Values")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Part 13 — Detect Duplicate Records

STUDENT GUIDE

`duplicated()` identifies duplicate rows. By default, pandas keeps the first occurrence and marks later identical copies as True. Expected result: 3 duplicate copies corresponding to BU003, BU008, and BU012.

```
duplicates = df.duplicated()

print("Number of duplicate rows:", duplicates.sum())
```

Part 14 — Display All Records Involved in Duplication

STUDENT GUIDE

`keep=False` marks every row belonging to a duplicate group. You should see 6 rows: two copies each of BU003, BU008, and BU012. Remember: 3 duplicate copies means 6 rows are involved in the three duplicate groups.

```
df[df.duplicated(keep=False)]
```

Part 15 — Show Only the Repeated Copies

STUDENT GUIDE

`duplicated()` without `keep=False` identifies only the later copies—the rows that would normally be removed by `drop_duplicates()`.

```
df[df.duplicated()]
```

Part 16 — Remove Exact Duplicate Records

STUDENT GUIDE

After confirming that these are exact duplicates, remove the repeated copies. Use `.copy()` because we will continue modifying `df_clean`. This also prevents pandas `SettingWithCopyWarning` when later changing columns. Expected result: 18 → 15 records, with 3 removed.

```
df_clean = df.drop_duplicates().copy()

print("Records before removing duplicates:", len(df))
print("Records after removing duplicates:", len(df_clean))
print("Records removed:", len(df) - len(df_clean))
```

Part 17 — Verify That Duplicates Were Removed

STUDENT GUIDE

Always verify a cleaning operation rather than assuming it worked. The expected number of remaining exact duplicates is zero.

```
print("Remaining duplicate rows:", df_clean.duplicated().sum())
```

Part 18 — Examine Categorical Values

STUDENT GUIDE

Categorical data often contain inconsistent values because different people enter the same category differently. `unique()` displays distinct values in `Sex`. Identify the different representations of Female and Male.

```
print("Unique values in Sex:")
print(df_clean["Sex"].unique())
```

Part 19 — Count Each Categorical Value

STUDENT GUIDE

`value_counts()` shows the frequency of each category. It helps identify inconsistent categories, rare categories, and possible data-entry errors.

```
print(df_clean["Sex"].value_counts())
```

QUESTION: How many different representations of Female exist? How many representations of Male exist?

Week 3 Coding Exercise — ML/NLP Data Preparation

Part 20 — Standardize the Sex Variable

STUDENT GUIDE

Standardize values that represent the same category. Female, female, F, and FEMALE become Female; Male and male become Male. Do not standardize blindly—first determine what each value actually means.

```
sex_mapping = {
    "Female": "Female",
    "female": "Female",
    "F": "Female",
    "FEMALE": "Female",
    "Male": "Male",
    "male": "Male"
}

df_clean["Sex"] = df_clean["Sex"].replace(sex_mapping)

print("Standardized Sex values:")
print(df_clean["Sex"].unique())
```

Part 21 — Verify the Standardized Categories

STUDENT GUIDE

After cleaning categorical data, verify the result. We should now have only Female and Male.

```
print(df_clean["Sex"].value_counts())
```

Part 22 — Examine Study Hours

STUDENT GUIDE

Inspect numerical variables for unusual values. Study_Hours contains 200, which is unrealistic for this practice scenario. An outlier is not automatically an error; investigate its context before deciding what to do.

```
print(df_clean["Study_Hours"].describe())
```

Part 23 — Visualize Study Hours

STUDENT GUIDE

A histogram shows the distribution of Study_Hours and should make the unusual value visually apparent. Some data-quality problems are easier to see in graphs than tables.

```
plt.figure(figsize=(8, 5))
plt.hist(df_clean["Study_Hours"].dropna(), bins=10)
plt.title("Distribution of Study Hours")
plt.xlabel("Study Hours")
plt.ylabel("Frequency")
plt.show()
```

Part 24 — Detect Outliers Using the IQR Method

STUDENT GUIDE

The Interquartile Range (IQR) is one method for identifying potential outliers. $IQR = Q3 - Q1$; lower bound = $Q1 -$

$1.5 \times \text{IQR}$; upper bound = $Q3 + 1.5 \times \text{IQR}$. Values outside the bounds are potential outliers, not automatically errors.

```
Q1 = df_clean["Study_Hours"].quantile(0.25)
Q3 = df_clean["Study_Hours"].quantile(0.75)

IQR = Q3 - Q1
```

Week 3 Coding Exercise — ML/NLP Data Preparation

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

Part 25 — Display Potential Outliers

STUDENT GUIDE

Identify observations outside the IQR boundaries. These are candidates for investigation, not automatic deletion.

```
outliers = df_clean[
    (df_clean["Study_Hours"] < lower_bound) |
    (df_clean["Study_Hours"] > upper_bound)
]

print("Potential outliers:")
display(outliers[["Student_ID", "Study_Hours"]])
```

Part 26 — Visualize Outliers Using a Boxplot

STUDENT GUIDE

A boxplot provides a visual representation of the distribution and potential outliers. Points outside the whiskers may require investigation.

```
plt.figure(figsize=(8, 5))
plt.boxplot(df_clean["Study_Hours"].dropna())
plt.title("Boxplot of Study Hours")
plt.ylabel("Study Hours")
plt.show()
```

Part 27 — Handle the Obvious Data-Entry Error

STUDENT GUIDE

For this exercise, assume Study_Hours = 200 is a data-entry error. Replace the incorrect value with NaN rather than deleting the entire student record, because the student's other information may still be valid.

```
df_clean.loc[df_clean["Study_Hours"] == 200, "Study_Hours"] = np.nan

print("Potential data-entry error replaced with NaN.")
```

Part 28 — Check Missing Values Again

STUDENT GUIDE

After replacing the suspected error with NaN, recheck the dataset. We now need to decide how to handle the missing Study_Hours values.

```
print(df_clean.isnull().sum())
```

Part 29 — Examine GPA

STUDENT GUIDE

GPA contains a missing value. Examine its descriptive statistics before choosing a method for handling it.

```
print(df_clean["GPA"].describe())
```

Week 3 Coding Exercise — ML/NLP Data Preparation

Part 30 — Median Imputation for GPA

STUDENT GUIDE

One common approach to missing numerical data is imputation. Here we use the median because it is less affected by extreme values than the mean. In a thesis, justify the chosen method based on the research context.

```
gpa_median = df_clean["GPA"].median()

print("GPA median:", gpa_median)

df_clean["GPA"] = df_clean["GPA"].fillna(gpa_median)
```

Part 31 — Impute Missing Study Hours

STUDENT GUIDE

Study_Hours now contains missing values because the unrealistic value of 200 was treated as an error. Replace missing Study_Hours using the median for this introductory exercise. A thesis should justify the selected preprocessing method.

```
study_hours_median = df_clean["Study_Hours"].median()

print("Study Hours median:", study_hours_median)

df_clean["Study_Hours"] = df_clean["Study_Hours"].fillna(
    study_hours_median
)
```

Part 32 — Check Whether Numerical Missing Values Remain

STUDENT GUIDE

After imputation, verify whether missing numerical values remain. Never assume that cleaning worked; always verify.

```
print(df_clean.isnull().sum())
```

Part 33 — Clean Text Data for NLP

STUDENT GUIDE

Text data usually requires preprocessing before NLP. Basic steps here are lowercasing, removing punctuation, and trimming spaces. In later activities, you will encounter tokenization, stopwords removal, stemming/lemmatization, TF-IDF, embeddings, and transformer representations.

```
import string

df_clean["Comment_Clean"] = (
    df_clean["Comment"]
    .str.lower()
    .str.strip()
    .str.translate(str.maketrans("", "", string.punctuation))
)
```

```
df_clean[["Comment", "Comment_Clean"]].head()
```

Part 34 — Compare Original and Cleaned Text

STUDENT GUIDE

Keep the original text for documentation and auditing while using the cleaned text for NLP processing. Never overwrite original research data without retaining a raw copy.

Week 3 Coding Exercise — ML/NLP Data Preparation

```
for original, cleaned in zip(
    df_clean["Comment"].head(5),
    df_clean["Comment_Clean"].head(5)
):
    print("ORIGINAL :", original)
    print("CLEANED :", cleaned)
    print("-" * 60)
```

Part 35 — Create a Simple NLP Feature: Word Count

STUDENT GUIDE

Word count demonstrates how raw text can be transformed into a numerical feature. This is a simple NLP feature, not a complete NLP model. Later you will learn Bag of Words, TF-IDF, embeddings, BERT, and transformers.

```
df_clean["Word_Count"] = (
    df_clean["Comment_Clean"]
    .str.split()
    .str.len()
)

df_clean[["Comment_Clean", "Word_Count"]].head()
```

Part 36 — Explore Academic Risk

STUDENT GUIDE

Academic_Risk is our target variable. Examine its class distribution before modeling. Understanding target distribution helps identify possible class imbalance.

```
print(df_clean["Academic_Risk"].value_counts())
```

Part 37 — Visualize Academic Risk

STUDENT GUIDE

A bar chart makes class distribution easier to understand. If one class greatly outnumbers another, class imbalance may affect model training and evaluation.

```
df_clean["Academic_Risk"].value_counts().plot(kind="bar")

plt.title("Academic Risk Distribution")
plt.xlabel("Academic Risk")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)
plt.show()
```

Part 38 — Examine Program Distribution

STUDENT GUIDE

Examine the number of students per academic program. This helps determine whether the sample is distributed

across programs or dominated by one group.

```
print(df_clean["Program"].value_counts())
```

Part 39 — Visualize Program Distribution

STUDENT GUIDE

Visualization makes differences in program counts easier to compare.

Week 3 Coding Exercise — ML/NLP Data Preparation

```
df_clean["Program"].value_counts().plot(kind="bar")

plt.title("Student Distribution by Program")
plt.xlabel("Program")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)
plt.show()
```

Part 40 — Analyze GPA Distribution

STUDENT GUIDE

A histogram shows how GPA values are distributed. Look for concentration, spread, and unusual values.

```
plt.figure(figsize=(8, 5))
plt.hist(df_clean["GPA"], bins=8)
plt.title("GPA Distribution")
plt.xlabel("GPA")
plt.ylabel("Frequency")
plt.show()
```

Part 41 — Analyze Attendance Distribution

STUDENT GUIDE

Attendance is another numerical variable. Examine its distribution before attempting to establish relationships with other variables.

```
plt.figure(figsize=(8, 5))
plt.hist(df_clean["Attendance"], bins=8)
plt.title("Attendance Distribution")
plt.xlabel("Attendance (%)")
plt.ylabel("Frequency")
plt.show()
```

Part 42 — Examine Attendance vs. GPA

STUDENT GUIDE

A scatter plot helps explore the relationship between two numerical variables. Compare Attendance and GPA. A visible relationship does not automatically prove causation.

```
plt.figure(figsize=(8, 5))
plt.scatter(df_clean["Attendance"], df_clean["GPA"])
plt.title("Attendance vs. GPA")
plt.xlabel("Attendance (%)")
plt.ylabel("GPA")
plt.show()
```

QUESTION: Does higher attendance appear to be associated with a particular GPA pattern?

Part 43 — Compare GPA by Academic Risk

STUDENT GUIDE

Grouping allows comparison of numerical variables across categories. Calculate average GPA for each Academic_Risk class. This is exploratory analysis, not proof that GPA causes academic risk.

```
gpa_by_risk = df_clean.groupby("Academic_Risk")["GPA"].mean()

print(gpa_by_risk)
```

Week 3 Coding Exercise — ML/NLP Data Preparation

QUESTION: Is there an apparent difference in GPA between students classified as "Yes" and "No" for academic risk?

Part 44 — Compare Study Hours by Academic Risk

STUDENT GUIDE

Compare average Study_Hours between risk groups to explore whether study time differs across

```
categories. study_hours_by_risk = df_clean.groupby("Academic_Risk")["Study_Hours"].mean()

print(study_hours_by_risk)
```

Part 45 — Create a Summary Table

STUDENT GUIDE

Create a compact summary of GPA, Study_Hours, and Attendance by Academic_Risk. Summary tables are useful for interpreting patterns and reporting descriptive results.

```
summary = df_clean.groupby("Academic_Risk") [
    ["GPA", "Study_Hours", "Attendance"]
].mean()

summary
```

Part 46 — Identify Features and Target

STUDENT GUIDE

Before supervised ML, identify features (X), target (y), text variables, and identifiers. Features are inputs used for prediction; the target is what the model predicts. Student_ID is normally not a predictive feature because it is only an identifier.

```
features = [
    "Age",
    "Sex",
    "Program",
    "Study_Hours",
    "Attendance",
    "GPA"
]

target = "Academic_Risk"
text_variable = "Comment_Clean"
identifier = "Student_ID"

print("Features:", features)
print("Target:", target)
```

```
print("Text Variable:", text_variable)
print("Identifier:", identifier)
```

Part 47 — Display the ML Dataset

STUDENT GUIDE

Create X and y to organize the data for a potential supervised ML task. This is data organization, not model training yet.

```
X = df_clean[features]
y = df_clean[target]
```

Week 3 Coding Exercise — ML/NLP Data Preparation

```
print("Feature dataset:")
display(X)

print("\nTarget:")
display(y)
```

Part 48 — Examine the Target Classes

STUDENT GUIDE

Academic_Risk contains Yes and No, so this is a binary classification problem. Binary classification has two classes; multiclass classification has more than two; multilabel classification can assign multiple labels to one record.

```
classes = df_clean[target].unique()

print("Target classes:")
print(classes)

print("\nNumber of classes:", len(classes))
```

Part 49 — Examine Correlations Among Numerical Variables

STUDENT GUIDE

Correlation measures the degree to which two numerical variables move together. Values range from -1 to +1. Correlation does not establish causation.

```
numerical_columns = [
    "Age",
    "Study_Hours",
    "Attendance",
    "GPA"
]

correlation_matrix = df_clean[numerical_columns].corr()

correlation_matrix
```

Part 50 — Visualize the Correlation Matrix

STUDENT GUIDE

A visual correlation matrix makes relationships easier to inspect. Identify stronger positive or negative relationships, but remember that correlation does not mean causation.

```
plt.figure(figsize=(8, 6))

plt.imshow(correlation_matrix, interpolation="nearest")
```

```
plt.colorbar()

plt.xticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns,
    rotation=45
)

plt.yticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns
)

plt.title("Correlation Matrix")

plt.tight_layout()
plt.show()
```

Week 3 Coding Exercise — ML/NLP Data Preparation

Part 51 — Generate a Basic Data Quality Report

STUDENT GUIDE

A data-quality report documents the dataset structure, missing values, data types, and unique values. This can support the methodology and preprocessing documentation in a thesis.

```
quality_report = pd.DataFrame({
    "Data_Type": df_clean.dtypes.astype(str),
    "Missing_Values": df_clean.isnull().sum(),
    "Unique_Values": df_clean.nunique()
})

quality_report
```

Part 52 — Check the Final Dataset

STUDENT GUIDE

Before saving, verify duplicates, missing values, standardized categories, the addressed outlier, cleaned text, and required variables. This is a final quality check before modeling.

CHECK: 1. Are duplicate records gone? 2. Are missing numerical values handled? 3. Are categorical values standardized? 4. Has the obvious outlier/data-entry error been addressed? 5. Is the text cleaned? 6. Are the important variables still present?

```
print("Dataset dimensions:", df_clean.shape)

print("\nRemaining duplicates:",
      df_clean.duplicated().sum())

print("\nRemaining missing values:")
print(df_clean.isnull().sum())

print("\nSex categories:")
print(df_clean["Sex"].unique())

print("\nFinal dataset:")
display(df_clean)
```

Part 53 — Save the Clean Dataset

STUDENT GUIDE

Save the processed dataset separately from the raw dataset. Never overwrite the original research data. Keeping

raw and clean versions supports reproducibility and auditing.

```
df_clean.to_csv(
    "week3_clean_dataset.csv",
    index=False
)

print("Clean dataset saved as:")
print("week3_clean_dataset.csv")
```

Week 3 Coding Exercise — ML/NLP Data Preparation

Part 54 — Reload the Clean Dataset

STUDENT GUIDE

Reloading the processed file demonstrates how a cleaned dataset can become the input for the next stage of the ML/NLP workflow.

```
final_df = pd.read_csv("week3_clean_dataset.csv")

print("Clean dataset successfully reloaded.")

display(final_df.head())
```

Part 55 — Final Verification

STUDENT GUIDE

Perform the final quality check. Verify the record count, variable count, duplicate count, missing values, and standardized categories before moving to baseline model development.

Before proceeding to model development, verify: ✓ Dataset exists ✓ Correct number of records ✓ No exact duplicates ✓ Missing values handled ✓ Categorical values standardized ✓ Numerical variables checked ✓

Text preprocessing completed

```
print("=" * 60)
print("FINAL DATASET QUALITY CHECK")
print("=" * 60)

print("Number of records:", len(final_df))
print("Number of variables:", len(final_df.columns))

print(
    "Duplicate records:",
    final_df.duplicated().sum()
)

print("\nMissing values:")
print(final_df.isnull().sum())

print("\nSex categories:")
print(final_df["Sex"].unique())

print("\nDataset status:")
print("Data preparation completed.")
```

Part 56 — Optional: Create a Complete Automated Summary

STUDENT GUIDE

This automated summary provides a compact overview of the cleaned dataset. Such automation becomes useful with larger datasets where manually checking every variable is impractical.

```
print("=" * 70)
print("WEEK 3 DATA PREPARATION SUMMARY")
print("=" * 70)

print(f"Number of records : {len(final_df)}")
print(f"Number of variables : {len(final_df.columns)}")

print(
    f"Duplicate records : "
    f"{final_df.duplicated().sum()}"
)
```

Week 3 Coding Exercise — ML/NLP Data Preparation

```
print(
    f"Total missing values : "
    f"{final_df.isnull().sum().sum()}"
)

print(
    f"Number of programs : "
    f"{final_df['Program'].nunique()}"
)

print(
    f"Number of target classes: "
    f"{final_df['Academic_Risk'].nunique()}"
)

print(
    f"Text records : "
    f"{final_df['Comment_Clean'].notnull().sum()}"
)

print("=" * 70)
print("Dataset is ready for the next ML/NLP activity.")
print("=" * 70)
```

Part 57 — Week 3 Independent Challenge

STUDENT GUIDE

Apply the techniques learned independently. Do not simply copy the earlier commands. Modify the code and explain your decisions. Identify numerical/categorical/text variables, target, record count, missing values, duplicates, inconsistent values, outliers, cleaning decisions, at least three visualizations, descriptive statistics, two relationships, ML readiness, ethical/privacy concerns, and how Comment could support an NLP task. Save your final cleaned dataset.

```
# Suggested starting point:
#
# 1. Identify variables
# 2. Inspect data quality
# 3. Clean the dataset
# 4. Perform EDA
# 5. Identify features and target
# 6. Assess ML/NLP readiness
# 7. Document ethical/privacy issues
# 8. Save the final dataset
#
# Write your own code below.

# YOUR CODE HERE
```

Week 3 Student Reflection

STUDENT GUIDE

Answer the following questions after completing the coding activity. Your answers should explain the reasoning behind your preprocessing and EDA decisions, not merely describe the Python commands.

1. What was the most serious data-quality problem you found?
 2. Why should duplicate records be removed before training an ML model?
 3. Why should researchers not automatically delete every outlier?
- Week 3 Coding Exercise — ML/NLP Data Preparation
4. Why were Sex values such as F, female, and Female standardized?
 5. Why is it important to keep the raw dataset?
 6. Which variables could be used as ML features?
 7. Which variable is the ML target?
 8. Which variable can be used for NLP?
 9. What ethical or privacy concerns might exist if this were a real student dataset?
 10. What additional preprocessing would be necessary before training an actual ML/NLP model?

Week 3 → Week 4 Connection

Week 3 prepares the dataset; Week 4 uses that prepared data to build and evaluate a baseline model.

Week 3: Data Collection → Inspection → Data Quality Assessment → Cleaning → EDA → Feature/Target Identification → NLP Preparation

Week 4: Train/Test Split → Baseline Algorithm → Model Training → Prediction → Accuracy/Precision/Recall/F1 → Baseline Interpretation

Expected Learning Evidence

- A cleaned dataset with duplicates, missing values, inconsistent categories, and an obvious data-entry error appropriately addressed.
- At least three meaningful EDA visualizations.
- Identification of ML features, target, textual variable, and identifier.
- Basic NLP text preprocessing and a simple text-derived feature.
- A documented data-quality assessment.
- Written reflection on privacy, bias, ethics, and model readiness.

Week 3 Coding Exercise — ML/NLP Data Preparation